



Módulo VI

Coleções em Java

Objetivo

Apresentar as principais classes de coleção de Java

Introduzir os conceitos de interface e realização

Créditos

Autor

Prof. Alessandro Cerqueira
(alessandro.cerqueira@hotmail.com)

Exercício

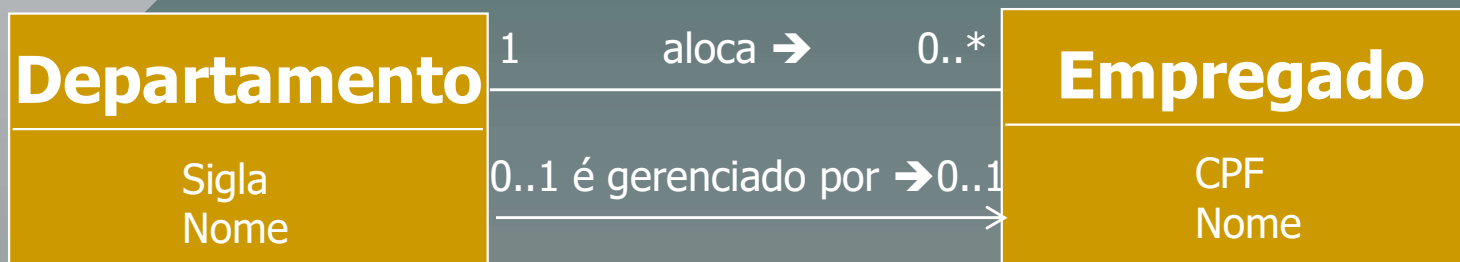
“Uma empresa deseja guardar em seu banco de dados quais são os seus empregados e quais são seus departamentos. Todo empregado tem CPF e Nome; Todo departamento possui sigla e nome. Todo empregado deve estar alocado em um departamento e todo departamento deve ter um empregado que é o seu gerente.”

Pede-se:

- **Modelo Conceitual** expresso através de um Diagrama de Classes
- **Modelo Lógico** expresso através de um Esquema Relacional

Coleções

- Considere o seguinte exemplo:



- Ao codificarmos estas classes, precisamos decidir como iremos implementar os relacionamentos em Java. Para isto, será necessário colocar novos atributos nas classes. Estes novos atributos são chamados de Atributos de Relacionamento ou Atributos Implícitos.
- Regra Geral:
 - Para cada **associação** ou **agregação navegável** presente na classe, adicionamos um atributo de relacionamento.
- Para relacionamentos **unários** colocamos um atributo cujo tipo deverá ser *“ponteiro para um objeto da classe XXX”*
- A grande questão é como são implementados os relacionamentos **n-ários**. Uma primeira idéia é utilizarmos *arrays* para isto.

Coleções

```
public class Departamento {

    private String sigla;
    private String nome;
    private Empregado[] listaEmpregados;
    private Empregado gerente;

    public Departamento(String sigla, String nome) {
        this.sigla = sigla;
        this.nome = nome;
        this.listaEmpregados = new Empregado[100];
    }

    public void addEmpregado(Empregado novoEmp) {
        ...
    }

    public void removeEmpregado(Empregado exEmp) {
        ...
    }

    ...
}
```

```
public class Empregado {

    private String cpf;
    private String nome;
    private Departamento depto;

    public Empregado(String cpf, String nome,
                     Departamento depto) {
        this.cpf = cpf;
        this.nome = nome;
        this.setDepto(depto);
    }

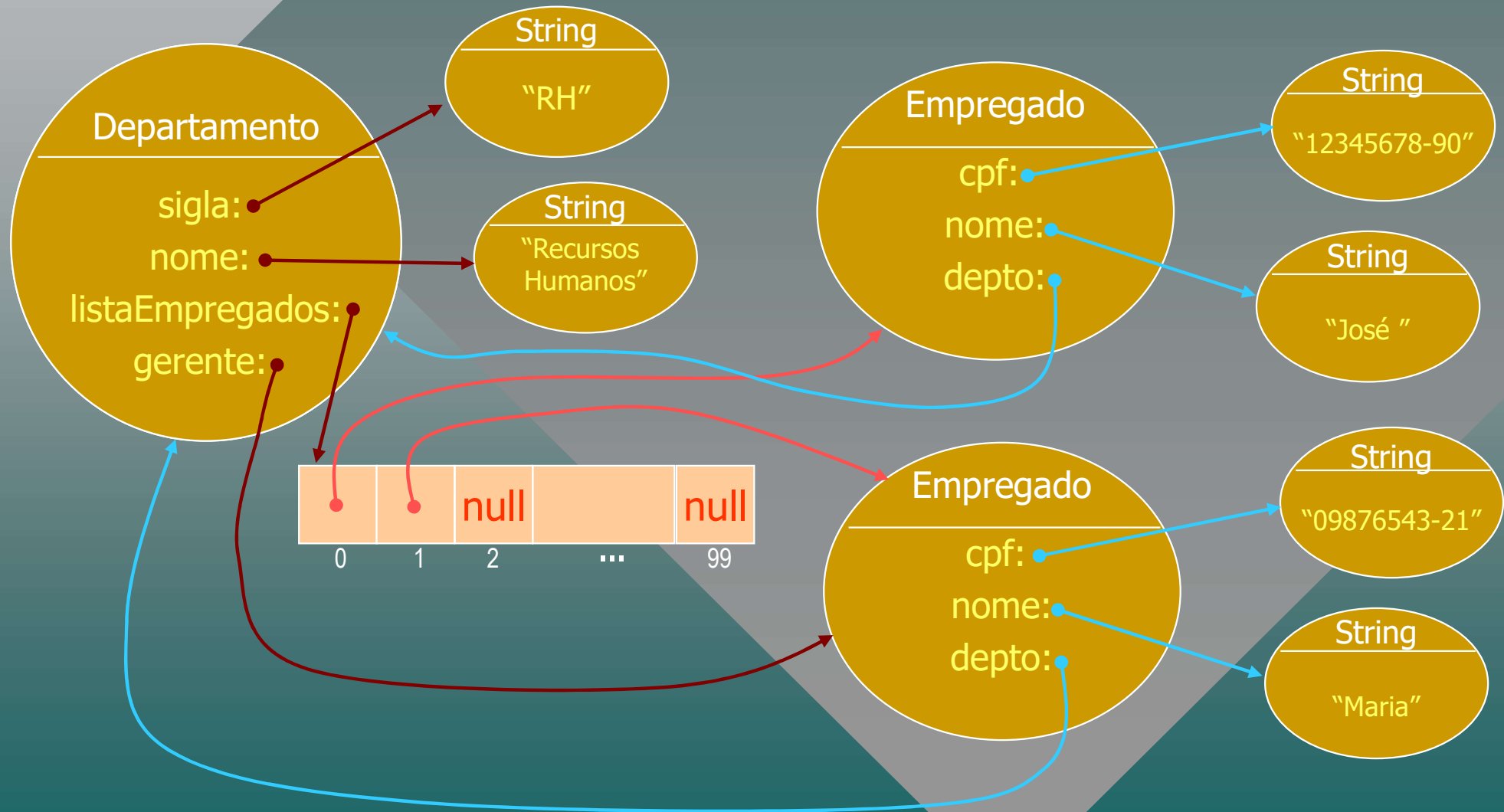
    public void setDepto(Departamento depto){
        ...
    }

    ...
}
```

- Nota: Observe que as duas associações são navegáveis a partir de Departamento, mas na classe Empregado somente a associação “Aloca” é navegável; por isso é que temos somente um atributo de relacionamento em Empregado, enquanto que em Departamento temos dois.

Exemplo

Relacionamentos Bidirecionais



Métodos para Estabelecer Relacionamentos

- **Além** de colocarmos os atributos de relacionamento, **necessitamos colocar métodos que vão estabelecer os relacionamento entre os objetos** e, conseqüentemente, manipulando os atributos de relacionamento.
- Regras:
 - Caso o **relacionamento** seja **unário** (ou seja, se o **atributo de relacionamento não é uma coleção**), deveremos colocar um **método set___**.
 - Caso o **relacionamento** seja **n-ário** (ou seja, se o **atributo de relacionamento é uma coleção**), deveremos colocar os **métodos add___** e **remove___**.
- A implementação destes métodos deve levar em conta se o relacionamento é unidirecional ou bidirecional.

Métodos para Estabelecer Relacionamentos Bidirecionais

NA CLASSE EMPREGADO

```
public void setDepto(Departamento depto) {  
    // Se a referência para o departamento do Empregado é a mesma que  
    // está sendo recebida pelo parâmetro, não há necessidade de atualização  
    if(this.depto == depto)  
        return;  
    // Se o parâmetro é null, isto indica que o Empregado deve ser retirado  
    // do departamento.  
    if(depto == null) {  
        Departamento antigo = this.depto;  
        // Apago a referência antiga  
        this.depto = null;  
        // Solicito ao Departamento para retirar sua referência para o Empregado  
        antigo.removeEmpregado(this);  
    }  
    else {  
        // Se o Empregado já pertence a um Departamento, vou solicitar sua remoção  
        // para colocá-lo em outro Departamento.  
        if(this.depto != null)  
            this.depto.removeEmpregado(this);  
        // Estabeleço a referência  
        this.depto = depto;  
        // solicito ao Departamento para adicionar uma referência para o Empregado.  
        depto.addEmpregado(this);  
    }  
}
```

Métodos para Estabelecer Relacionamentos Bidirecionais

NA CLASSE DEPARTAMENTO

```
public void addEmpregado(Empregado novoEmp) {  
    // Se a referência para o Empregado já está presente na lista de  
    // Empregados do Departamento, não há necessidade de atualização  
    if(this.listaEmpregados.contains(novoEmp))  
        return;  
    // Solicito ao objeto de coleção referenciado por this.listaEmpregados  
    // que adicione uma referência para o novo empregado  
    this.listaEmpregados.add(novoEmp);  
    // Solicito ao Empregado que referencie o seu novo departamento  
    novoEmp.setDepto(this);  
}  
  
public void removeEmpregado(Empregado exEmp) {  
    // Se a referência para o Empregado não está presente na lista de  
    // Empregados do Departamento, não há necessidade de atualização  
    if( ! this.listaEmpregados.contains(exEmp))  
        return;  
    // Solicito ao objeto de coleção referenciado por this.listaEmpregados  
    // que remova a referência para o ex-empregado  
    this.listaEmpregados.remove(exEmp);  
    // Solicito ao Empregado que deixe de referenciar o departamento  
    exEmp.setDepto(null);  
}
```

Exemplo Relacional

- A idéia de adicionarmos atributos para a implementação de relacionamentos também ocorre com o uso do modelo relacional através do conceito de **Chave Estrangeira**.
- Lá, entretanto, não há a diferença entre relacionamentos unidirecionais e bidirecionais.
- Para descobrirmos dados entre as entidades relacionadas, nós necessitamos realizar operações de **JUNÇÃO**.

DEPARTAMENTO		
sigla	nome	gerente
JR	Jurídico	901-2
LG	Logística	123-4

EMPREGADO		
cpf	nome	depto
123-4	José	LG
567-8	Maria	LG
901-2	Patrícia	JR
345-6	João	JR



Uma Forma de Execução da Junção

```
SELECT D.Nome, E.Nome (3°- projeção)
FROM Departamento D, Empregado E (1°- Prod.Cartesiano)
WHERE D.gerente = E.Cpf (2°- Seleção)
```



D.sgla	D.nome	D.gerente	E.cpf	E.nome	E.depto
JR	Jurídico	901-2	123-4	José	LG
JR	Jurídico	901-2	567-8	Maria	LG
JR	Jurídico	901-2	901-2	Patrícia	JR
JR	Jurídico	901-2	345-6	João	JR
LG	Logística	123-4	123-4	José	LG
LG	Logística	123-4	567-8	Maria	LG
LG	Logística	123-4	901-2	Patrícia	JR
LG	Logística	123-4	345-6	João	JR

Coleções

- A implementação de atributos n-ários com *arrays* não é adequada. (*Arrays ou apresentam desperdício ou apresentam falta de posições de memória*).
- Uma idéia melhor é utilizar a classe *java.util.Vector*.
- *Comparação entre Array x Vector*
 - **Arrays**
 - Tamanho fixo e definido na criação do array.
 - Procura pelo índice.
 - Elementos do mesmo tipo.
 - Arrays de tipos primitivos ou de ponteiros para instâncias de uma classe específica.
 - **Objetos Vector**
 - O tamanho inicial é definido quando o vector é criado. Este pode ser modificado manualmente ou automaticamente.
 - Procura pelo índice ou conteúdo.
 - Elementos de qualquer classe.
 - Vectors de ponteiros para instâncias de uma classe específica

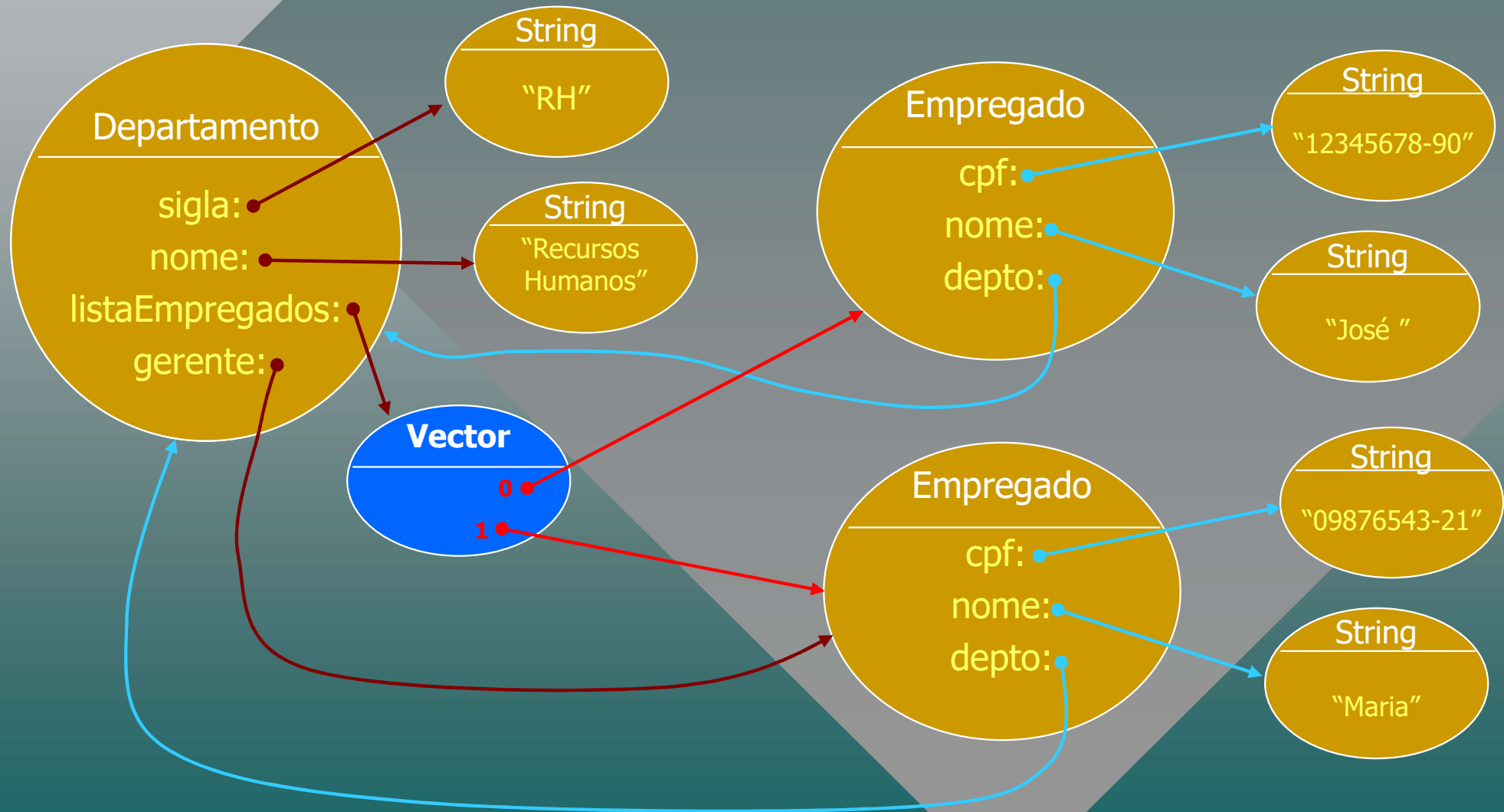
Coleções

```
public class Departamento {  
  
    private String      sigla;  
    private String      nome;  
    private Vector      listaEmpregados;  
    private Empregado    gerente;  
  
    public Departamento(String sigla, String nome) {  
        this.sigla = sigla;  
        this.nome = nome;  
        this.listaEmpregados = new Vector( );  
    }  
    public void addEmpregado(Empregado novoEmp) {  
        ...  
    }  
    public void removeEmpregado(Empregado exEmp){  
        ...  
    }  
    ...  
}
```

```
public class Empregado  
{  
    private String      cpf;  
    private String      nome;  
    private Departamento depto;  
  
    public Empregado(String cpf, String nome,  
                       Departamento depto) {  
        this.cpf = cpf;  
        this.nome = nome;  
        this.setDepto(depto);  
    }  
  
    public void setDepto(Departamento depto) {  
        ...  
    }  
    ...  
}
```

Exemplo com Vector

Relacionamentos Bidirecionais



Coleções

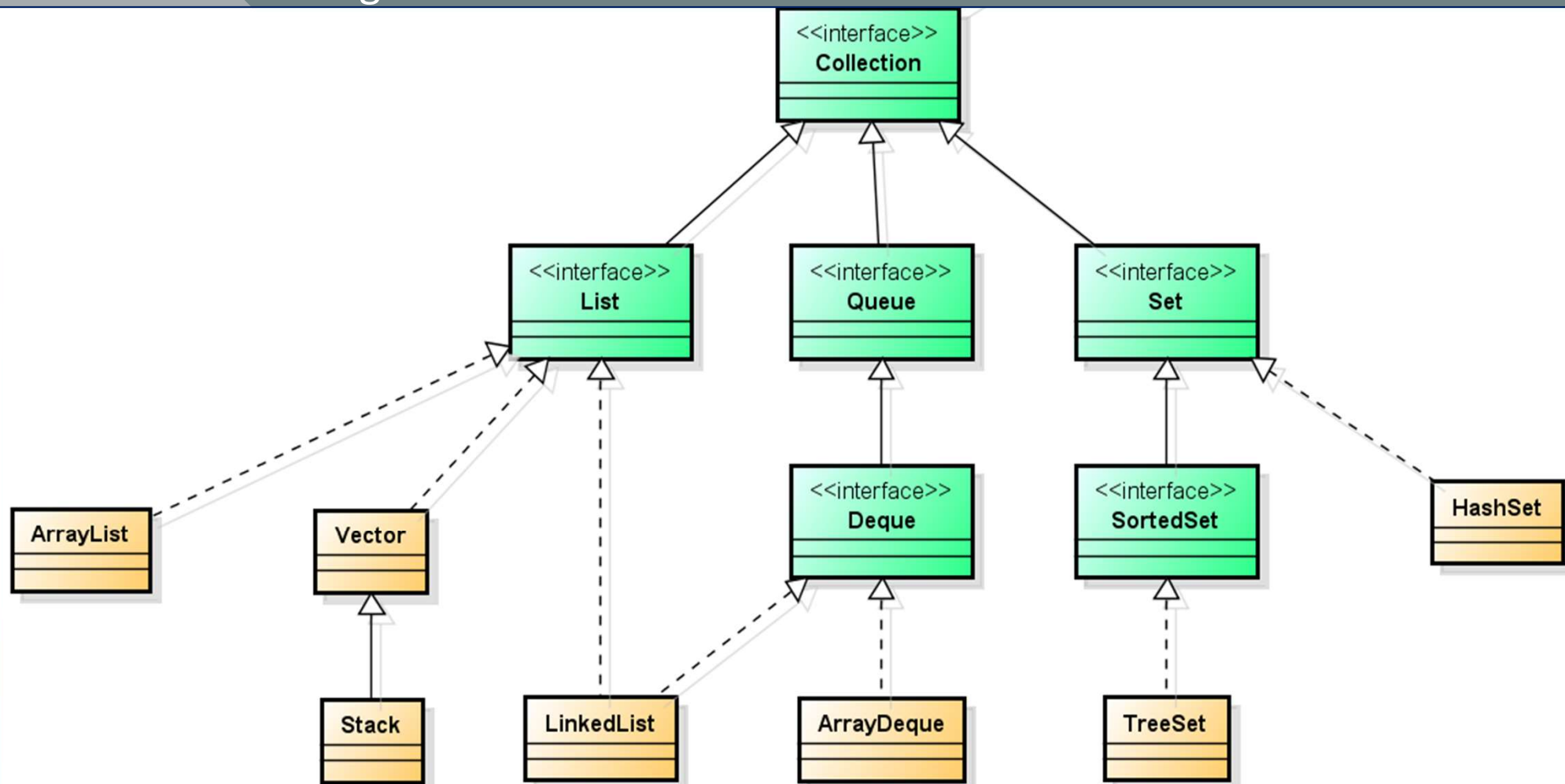
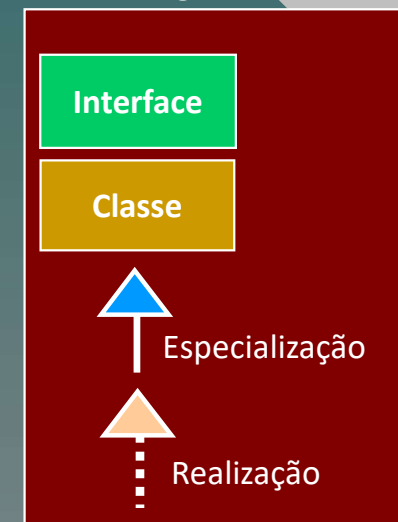
- Alguns métodos úteis da classe Vector:
 - **Vector()** : Cria uma nova instância sem elementos e com capacidade inicial para 10 elementos.
 - **Vector(int capacidade)** : Cria uma nova instância sem elementos e com capacidade inicial especificada pelo parâmetro.
 - **Vector(int capacidade, int fator)** : Cria uma nova instância sem elementos e com capacidade inicial / fator de crescimento especificados pelos parâmetros.
 - **add(Object elem)** : adiciona um objeto no final do Vector.
 - **add(int índice, Object elem)** : adiciona um objeto na posição especificada que deve estar entre 0 e o tamanho do Vector.
 - **setElementAt(Object elem, int índice)** : coloca o objeto em determinada posição, removendo o objeto que estiver lá anteriormente.
 - **remove(Object elem)** : remove a primeira ocorrência do objeto no Vector. Retorna *true* caso tenha sido possível a remoção.
 - **remove (int índice)** : remove o objeto na posição especificada.
 - **size()** : retorna o número de objetos presentes no Vector.

Coleções

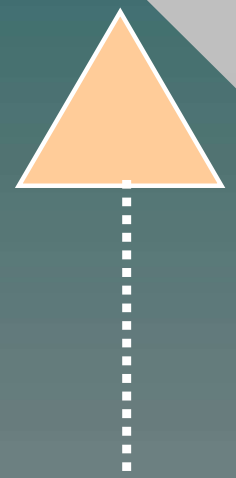
- **get(int índice)** : Retorna o objeto presente na posição especificada.
- **firstElement()** : Retorna o objeto presente na primeira posição.
- **lastElement()** : Retorna o objeto presente na última posição.
- **indexOf(Object elem)** : retorna o índice da primeira ocorrência do objeto no Vector ou -1 se o objeto não estiver presente.
- **indexOf(Object elem, int índice)** : retorna o índice da primeira ocorrência do objeto no Vector a partir da posição especificada. Retorna -1 se o objeto estiver presente.
- **lastIndexOf(Object elem)** : retorna o índice da última ocorrência do objeto no Vector ou -1 se o objeto não estiver presente.
- **lastIndexOf(Object elem, int índice)** : retorna o índice da última ocorrência do objeto no Vector a partir da posição especificada. Retorna -1 se o objeto estiver presente.
- **contains(Object elem)** : Informa se o objeto está presente no Vector.

Coleções no JDK 1.2

- Com o advento do **jdk 1.2**, A classe **Vector** está sendo evitada pois os seus métodos são **synchronized** (não permite acesso concorrente aos dados do Vector).
- A estrutura das classes de coleção (**pacote java.util**) está colocada da seguinte forma:



Conceito de Interface



- Conceito presente em Java e na UML
- Definição
 - Uma interface pode ser vista como a especificação de um **contrato de prestação de serviços** que pode ser **realizado (ou implementado)** por uma ou mais classes.
- Propriedades
 - Se assemelha a uma **classe abstrata**
 - **Não apresenta atributos** (somente constantes)
 - **Não apresenta código** para os seus métodos*
 - Possui somente as **assinaturas dos métodos** que deverão ser codificados pelas classes que implementarem a interface.*
 - **Obs*** - Se um método não apresenta código é só possui a sua assinatura, então ele é um **método abstrato**.
- Geralmente opta-se por uma interface (e não por uma classe abstrata) quando não se sabe como implementar o serviço ou quando o serviço pode ser implementado de várias maneiras.

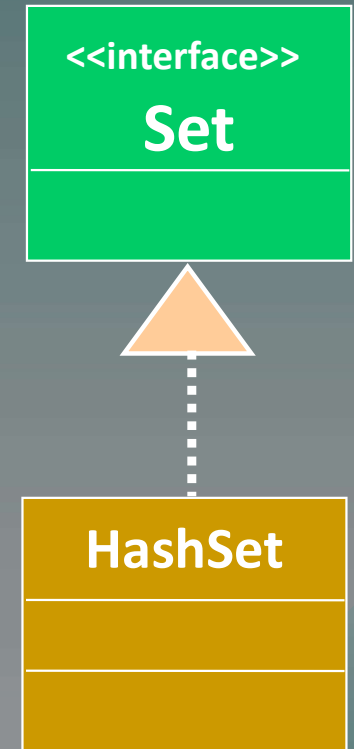
Conceito de Interface

- Realização

- Relacionamento definido na UML e em Java que indica que uma determinada **classe** **irá realizar** o **serviço** especificado por uma **interface**. Em Java costumamos popularmente chamar a realização de **implementação**.
- **Realizar** (ou **implementar**) uma interface significa que **a classe deverá implementar todos os métodos assinados na interface**. Se algum método ficar de fora, o compilador irá considerar a **classe abstrata**.
- Uma classe pode implementar quantas interfaces quiser.
- O conceito de polimorfismo se aplica aos objetos de uma classe que implementam uma interface
- Ex:

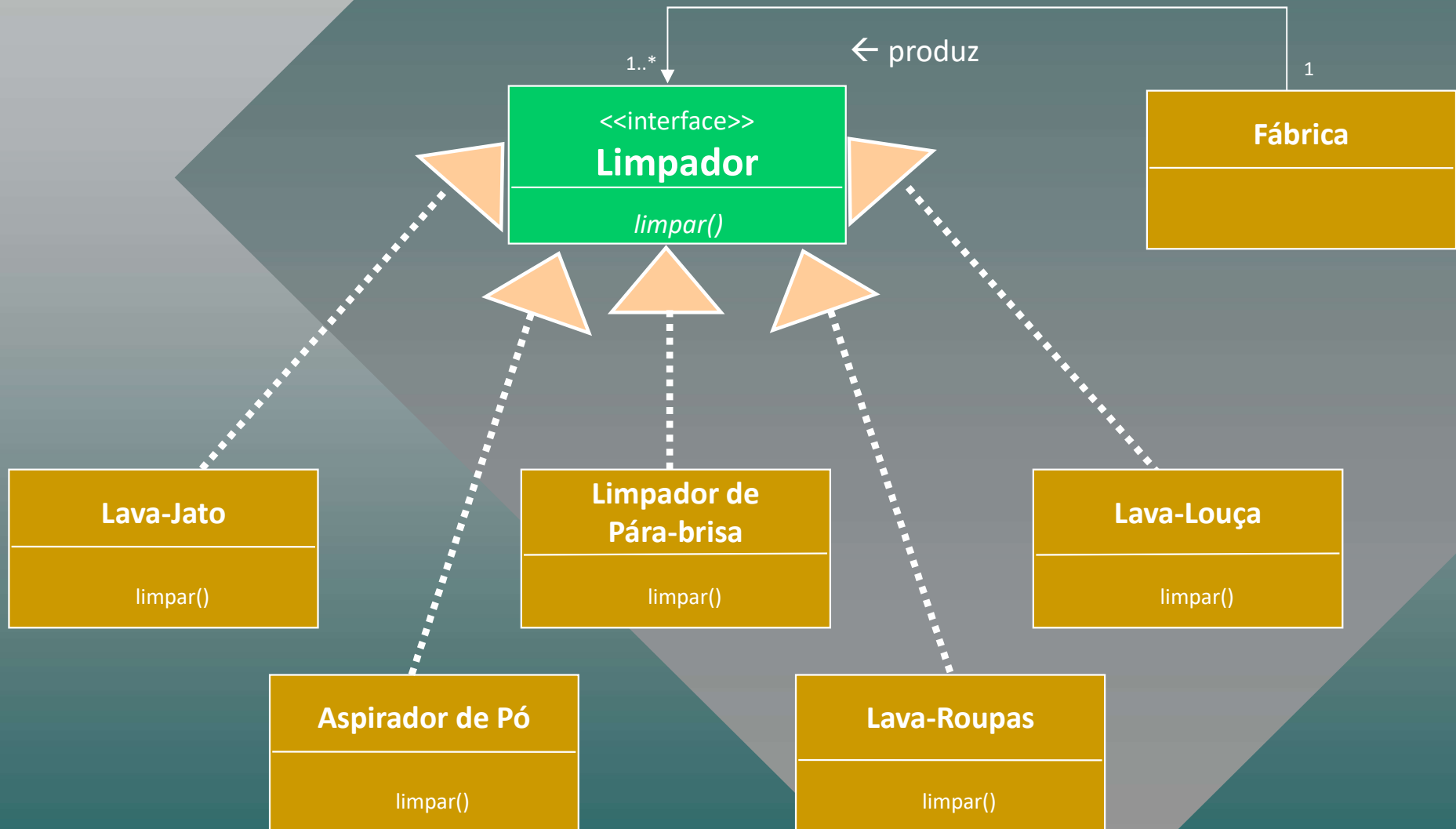
```
public class HashSet implements Set { ... }
```

Assim objetos HashSet também podem ser vistos como objetos Set



Interface e Realização

Exemplo



OBS: É melhor deixar Limpador como Interface pois esta apresenta todas as características de uma interface e assim estamos livres para deixar as classes que implementam esta interface como especializações de uma outra classe (ex. Equipamento Elétrico). Lembre-se que em Java uma classe só pode ser especialização direta de uma outra classe!

Interface e Realização

Exemplo

- Objetos **Lava-Jato**, **Aspirador de Pó**, **Limpador de Pára-Brisa**, **Lava-Roupas** e **Lava-Louça** apresentam algumas propriedades em comum. Todas são equipamentos produzidos por uma fábrica e que realizam o serviço de limpeza. Suponha que a execução da limpeza seja executada através do método **limpar()**.
- Para caracterizar que estas classes são limpadores, poderíamos criar uma **classe abstrata** chamada **Limpador** com uma **associação com Fábrica** e apresentando o método **limpar()**.
- Entretanto, observe que a suposta classe **Limpador** não apresentaria atributos e seu **único método é abstrato** (a forma como cada classe irá implementar o método **limpar()** é diferente e não conseguimos produzir uma implementação que valha para todas as suas especializações).

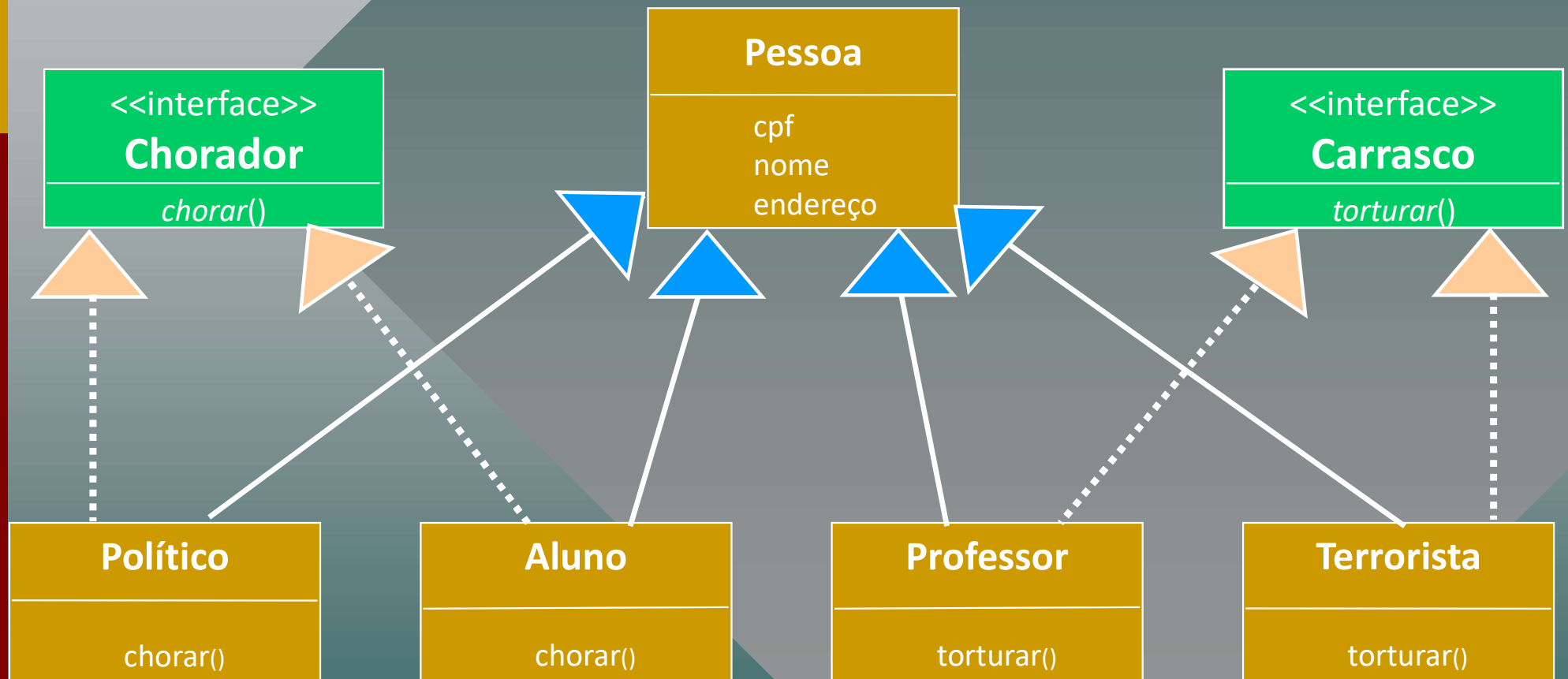
Interface e Realização

Exemplo

- Assim, é melhor caracterizar a classe Limpador como interface pois apresenta todas as características da mesma e, assim, permitimos que as suas realizações possam ser especializações de uma outra classe (lembre-se que em Java não há herança múltipla).
- Contudo, observe que **na realização há a presença da característica do polimorfismo**; pois os objetos **Fábrica** vêem os objetos **Lava-Jato**, **Aspirador de Pó**, **Limpador de Pára-Brisa**, **Lava-Roupas** e **Lava-Louça** como objetos da interface Limpador.

Interface e Realização

Exemplo (talvez um pouco mais lúdico)



Codificando...

```
public interface Carrasco {  
    public abstract void torturar();  
    // ou somente void torturar();  
    // pois sempre é public e abstract  
}
```

```
public class Professor extends Pessoa  
    implements Carrasco {  
    ...  
  
    public void torturar() {  
        System.out.println("Se não estudar, vou te reprovar!!!")  
    }  
  
    ...  
}
```

Coleções no JDK 1.2

- Interface *Collection*

- Uma *collection* representa um serviço capaz de apontar para um conjunto de objetos que não estão necessariamente postos em uma ordem de posição arbitrada pelo programador e que pode apresentar duplicações.

- Métodos Importantes:

boolean `add(Object o)` - *Adiciona um objeto à coleção*

boolean `addAll(Collection c)` - *Adiciona uma coleção de objetos à coleção*

void `clear()` – *Remove todos os objetos da coleção*

boolean `contains(Object o)` – *Informa se um objeto está na coleção*

Iterator `iterator()` – *Retorna um iterator para percorrer a coleção*

boolean `remove(Object o)` – *Retira um objeto da coleção*

boolean `removeAll(Collection c)` – *Retira uma coleção de objetos da coleção*

int `size()` – *Informa o número de elementos da coleção*

Coleções no JDK 1.2

- Interface *List*

- Especialização de Collection; Uma lista representa um serviço capaz de apontar para **um conjunto de objetos** que **estão colocados em uma ordem de posição** arbitrada pelo programador e que **pode apresentar duplicação de elementos**.
- Assim, o programador determinar que elemento ficará na posição 0, na posição 1, etc.

- **Métodos Importantes:**

Object **get**(int posicao) – *Retorna o objeto que está na posição indicada*

int **indexOf**(Object o) – *Retorna a posição onde o objeto está na coleção*

void **add**(int posicao, Object o) – *Insere um objeto em uma posição*

Object **set**(int posicao, Object o) – *Insere um objeto em uma posição, porém retirando o objeto anterior.*

List **subList**(int inicio, int fim) – *Retorna uma sublista*

➤ Os métodos de *Collection* também estão presentes!!!

Coleções do JDK 1.2

- Classe *ArrayList*
 - Implementação (ou realização) da interface *List*.
 - Seu comportamento está baseado em possuir um **array de referências para *Object*** que aponta para os elementos da lista.
 - Se o tamanho do array for insuficiente:
 - Cria-se um novo array com o tamanho maior;
 - Copia-se o conteúdo do antigo array para o novo;
 - Descarta-se o antigo (sofrerá o Garbage Collection).
 - **Bom** para recuperação aleatória por posição;
 - “**Ruim**” para inserção se realizar o redimensionamento do array a cada instante.
 - Para melhorar o desempenho, podemos utilizar o método **`ensureCapacity(tamanho)`**

Coleções do JDK 1.2

- *Classe `LinkedList`*
 - Seu comportamento está baseado em implementar uma *lista encadeada*
 - *Bom* para inserção;
 - “*Ruim*” para recuperação aleatória por posição já que há a necessidade de navegação pela lista. Entretanto, se a recuperação for iterativa (ou seja, posição a posição) não há problemas.
- *Classe `Vector`*
 - Comportamento igual ao do `ArrayList` porém o `Vector` é *synchronized*; ou seja, não permite acesso concorrente.

Coleções no JDK 1.2

- Interface Set

- Especialização de *Collection*; Um set representa um serviço capaz de apontar para um conjunto de objetos que não estão necessariamente colocados em uma ordem de posição arbitrada pelo programador e que não permite a duplicação de elementos.
- Métodos Importantes:
Os mesmos de *Collection*.

- Classe HashSet

- Implementação de Set. Para isto, utiliza internamente uma HashMap.
- Métodos Importantes:
HashSet(int capacidadeInicial)
HashSet(int capacidadeInicial, float fatorDeCarga)

Iterator

- Objeto que permite a recuperação seqüencial dos elementos de uma coleção.

```
Collection colecao;  
...  
for(Iterator it = colecao.iterator(); it.hasNext(); ) {  
    <Classe> elemento = (<Classe>)it.next();  
    ...  
}
```

- Métodos Importantes:

boolean hasNext() – *Informa se o iterator ainda pode retornar algum elemento.*

Object next() – *Retorna o próximo elemento a ser recuperado pelo iterator.*

Coleções no JDK 1.2

- Interface SortedSet

- Especialização de *Set*; Um SortedSet representa um serviço capaz de apontar para um conjunto de objetos sem duplicações cujos elementos estão dispostos ordenadamente segundo algum critério especificado.
- Para viabilizar a ordenação, é necessário que o SortedSet saiba o critério para ordenação. Para especificação do critério, devemos fazer com que os elementos do SortedSet implementem a interface *Comparable* ou passarmos para o SortedSet um objeto que implemente a interface *Comparator*.
- Métodos Importantes:
Os mesmos de *Set*.

- Classe TreeSet

- Implementa um SortedSet cuja complexidade de ordenação é $\text{Log}(n)$
- Métodos Importantes:
TreeSet() → Os elementos deverão implementar *Comparable*
TreeSet(*Comparator* comparador) → Utiliza um comparador para estabelecer a ordem

Comparable

- Se ao criar um TreeSet não indicamos um **Comparator**, ele considerará que as classes de seus elementos implementam a interface **Comparable**.
 - **Interface Comparable**
 - **int compareTo(Object o)**
 - Se o objeto que receber a mensagem (**this**) é **igual** ao objeto recebido como parâmetro, então o método deve retornar 0 (Zero)
 - Se o objeto que receber a mensagem (**this**) **vem primeiro** que o objeto recebido como parâmetro, então o método deve retornar um **número negativo**.
 - Se o objeto que receber a mensagem (**this**) **vem depois** que o objeto recebido como parâmetro, então o método deve retornar um **número positivo**.

Comparable

- Exemplo de Comparação com Strings

```
public class Empregado implements Comparable {  
    private String nome;  
  
    ...  
  
    public int compareTo(Object o) {  
        if(o instanceof Empregado) {  
            Empregado e = (Empregado)o;  
            return this.nome.compareTo(e.nome);  
        }  
        return -1;  
    }  
}
```

A classe String também é uma realização de Comparable!

Comparable

- Exemplo de Comparação Numérica

```
public class Empregado implements Comparable {  
    private int idade;  
    ...  
    public int compareTo(Object o) {  
        if(o instanceof Empregado) {  
            Empregado e = (Empregado)o;  
            return this.idade - e.idade;  
        }  
        return -1;  
    }  
}
```

Erros Vinculados à implementação do Comparable

- Se em uma classe colocarmos a indicação **"implements Comparable"** mas não adicionarmos o método **"compareTo"**, o compilador acusará **erro**.
- Da mesma forma, não adianta colocar o método **"compareTo"** e não colocarmos a indicação **"implements Comparable"**, pois quando o **TreeSet** utiliza o critério do **Comparable**, ele só permite que sejam adicionados Objetos que sejam **Comparable**. Assim, este erro só será percebido em tempo de execução.

Comparator

- Se ao criar um TreeSet passamos um Comparator, este determinará a ordem dos elementos utilizando o método **compare**.
 - **Interface Comparator**
 - **int compare(Object o1, Object o2)**
 - Se o objeto apontado por **o1** é **igual** ao objeto apontado por **o2**, então o método deve retornar 0 (Zero)
 - Se o objeto apontado por **o1** **vem primeiro** que o objeto apontado por **o2**, então o método deve retornar um **número negativo**.
 - Se o objeto apontado por **o1** **vem depois** que o objeto apontado por **o2**, então o método deve retornar um **número positivo**.

Comparator

- Exemplo de Comparator

```
public class ComparadorPorNome implements Comparator {  
    public int compare(Object o1, Object o2) {  
        if(o1 instanceof Empregado &&  
            o2 instanceof Empregado) {  
            Empregado e1 = (Empregado) o1;  
            Empregado e2 = (Empregado) o2;  
            return e1.getNome().compareTo(e2.getNome());  
        }  
        return -1;  
    }  
}
```

Operador *instanceof*

- Se quisermos **testar** um atributo ou variável local declarado como **ponteiro** para um objeto de uma superclasse está propriamente apontando para um objeto de uma de suas subclasses, devemos utilizar o operador **instanceof**.

```
// Elemento foi declarado como ponteiro para um
// objeto da classe Object
Object elemento = coleção.get(posição);
// Para sabermos se elemento na realidade está apontando
// para um objeto da classe String (subclasse), devemos
// utilizar o operador instanceof
if(elemento instanceof String) {
    ...
}
```

Casting e instanceof

- Observe que nos códigos exibidos temos a presença da seguinte linha:

```
Empregado e = (Empregado) o;
```

- Por que há a necessidade de colocarmos **(Empregado)** ?

- Java é uma linguagem fortemente tipada!

- Há **intensa verificação dos tipos** feita pelo compilador nas operações que codificamos.

- No caso de atribuições, o compilador verificará se o tipo à direita é **igual ou compatível** com o tipo à esquerda

- Ex: `int a = false;` // O tipo à direita (boolean) não é igual nem // compatível com o tipo à esquerda (int)

- A verificação também é feita na **passagem de parâmetros** (o tipo do parâmetro deve ser **igual ou compatível** com o especificado no método).

- Observe que no código, o tipo de **o** (elemento à direita) é “**ponteiro para um objeto da classe Object**” e o tipo de **e** (elemento à esquerda) é “**ponteiro para um objeto da classe Empregado**”.

Casting e instanceof

- (Continuação) Neste caso, o tipo à direita não é igual nem compatível com o tipo à esquerda (nem sempre um ponteiro para Object está apontando para um Empregado).
- Mas, por causa do teste realizado previamente com o **instanceof**, sabemos que **o** (ponteiro para Object) aponta para um objeto Empregado.
- Para o compilador não reclamar da operação (*Type Mismatch* – tipo incompatível), devemos solicitar que ele veja somente neste instante o ponteiro para Object como ponteiro para Empregado. A esta operação denominamos de **casting**.
- Se por acaso **o** não estivesse apontando para um Empregado, a JVM lançaria uma exceção chamada **ClassCastException** que abortaria a execução do programa.

Casting e instanceof

- Sempre precisaremos de indicar um casting na recuperação de elementos, pois todos os métodos de recuperação foram codificados para retornar uma referência para *Object*. Porém, na maioria das vezes, sabemos que eles pertencem a uma especialização de *Object*.
- Observe o seguinte caso:

```
String str = coleção.get(posição);
```

ptr para String

ptr para Object

- O compilador irá acusar erro pois o tipo à direita não é igual nem compatível com o tipo à esquerda. Mas se soubermos que *Coleção.get(posição)* sempre retornará uma String, devemos avisar isto para o compilador através de um *Casting*.

```
String str = (String) coleção.get(posição);
```

- Dica para o uso do *casting* em atribuições
 - Se o tipo da direita **não é igual ou compatível** com o tipo da esquerda, deveremos utilizar o Casting.

Coleções

Implementando Classes de Associação

- Devemos substituir a modelagem proposta pela implementação de duas associações ligando a classe de associação com as classes envolvidas no relacionamento.
- No exemplo abaixo, apesar de termos modelado a associação “**Envolve a Venda de**”, vamos substituí-la pelas associações em vermelho. Observe que as cardinalidades das extremidades são unárias e que as cardinalidades na classe de associação são trocadas em relação ao que existia antes (em azul).



Estabelecimento de Relacionamentos

Resumo

- Quando o relacionamento for Unário
 - Adicionamos um **atributo** cujo tipo será **ponteiro** para um **objeto da classe** <...>
 - Adicionamos um **método** **set**<...>
- Quando o relacionamento for N-ário
 - Adicionamos um **atributo** cujo tipo será **ponteiro** para um **objeto de coleção**
 - Adicionamos os **métodos** **add**<...> e **remove**<...>

Estabelecimento de Relacionamentos

Resumo

Estrutura do Método set_____

```
public void set[ATRIBUTO] ([CLASSE_APONTADA] objClasse) {  
    // Se [ATRIBUTO] aponta para o mesmo objeto apontado por objClasse  
    // não há necessidade de atualização  
    if(this.[ATRIBUTO] == objClasse)  
        return;  
  
    // Se o parâmetro é null, isto indica que os vínculos devem ser  
    // removidos  
    if(objClasse == null) {  
        [CLASSE_APONTADA] antigo = this.[ATRIBUTO];  
  
        // Apago a referência antiga  
        this.[ATRIBUTO] = null;  
  
        // Solicito ao objeto antigo que deixe de referenciar o this.  
        antigo.remove[CLASSE] (this); // se n-ário  
        OU  
        antigo.set[CLASSE] (null); // se unário  
    }  
}
```

Estabelecimento de Relacionamentos

Resumo

(cont.) Estrutura do Método set_____

```
else {  
    // Se o this já apontava para um objeto de [CLASSE_APONTADA],  
    // vamos fazer a remoção do vínculo antigo  
    if(this.[ATRIBUTO] != null)  
        this.[ATRIBUTO].remove[CLASSE](this); OU  
        this.[ATRIBUTO].set[CLASSE](null);  
  
    // Estabeleço a referência  
    this.[ATRIBUTO] = objClasse;  
  
    // solicito ao objClasse para referenciar o this.  
    objClasse.add[CLASSE](this); // se n-ário  
  
    OU  
    objClasse.set[CLASSE](this); // se unário  
}
```

Estabelecimento de Relacionamentos

Resumo

Estrutura do Método add_____

```
public void add[CLASSE_APONTADA] ([CLASSE_APONTADA] novo) {  
    // Se a referência já está presente na lista de [CLASSE_APONTADA]  
    // não há necessidade de atualização  
    if(this.lista[CLASSE_APONTADA]s.contains(novo))  
        return;  
  
    // Adicionando a referência na lista[CLASSE_APONTADA]s  
    this.lista[CLASSE_APONTADA]s.add(novo);  
  
    // Solicito a adição da referência no sentido inverso  
    novo.add[CLASSE](this); // se n-ário  
    OU  
    novo.set[CLASSE](this); // se unário  
}
```

Estabelecimento de Relacionamentos

Resumo

Estrutura do Método remove_____

```
public void remove[CLASSE_APONTADA] ([CLASSE_APONTADA] antigo) {  
    // Se a referência antiga já não está presente na lista  
    // não há necessidade de atualização  
    if( ! this.lista[CLASSE_APONTADA].contains(antigo) )  
        return;  
  
    // Removendo a referência antiga  
    this.lista[CLASSE_APONTADA].remove(antigo);  
  
    // Solicito a remoção da referência no sentido inverso  
    antigo.remove[CLASSE](this); // se n-ário  
    OU  
    antigo.set[CLASSE](null); // se n-ário  
}
```

Classe Collections

- É uma classe que contém vários métodos estáticos que fazem a manipulação dos objetos de coleção.

- Métodos Importantes:

- `static void copy(List l1, List l2)`
- `static void reverse(List l)`
- `static void sort(List l)`
- `static void sort(List l, Comparator cmp)`
- `static Collection unmodifiableCollection(Collection c)`
- `static List unmodifiableList(List c)`
- `static Collection synchronizedCollection(Collection c)`
- `static List synchronizedList(List c)`

Exemplo:

```
ArrayList lista = new ArrayList();
```

```
...
```

```
Collections.sort(lista); // Promoverá a ordenação da lista através de Comparable's
```

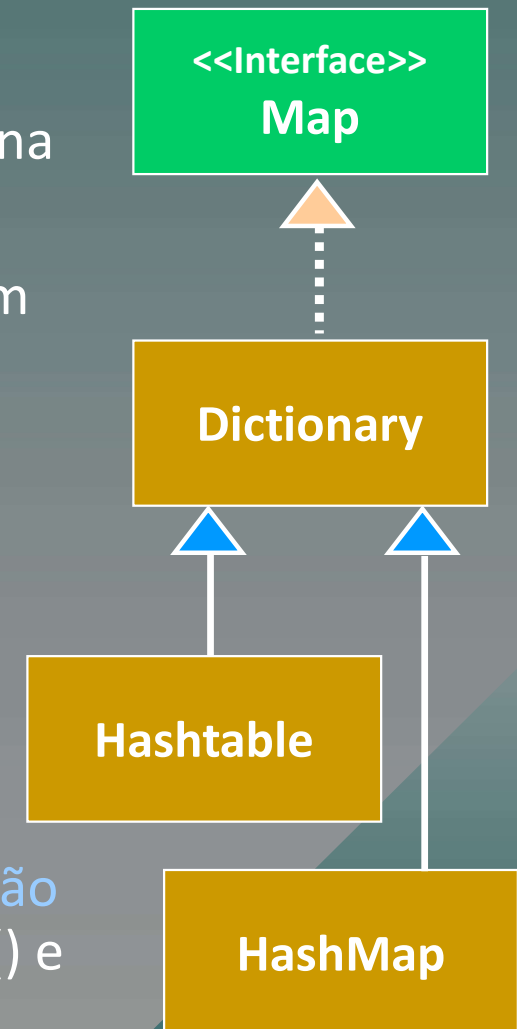
- Coleções Synchronized

- São aquelas que só podem ser manipuladas por uma única Thread a cada instante. A única exceção é a recuperação via iterator.

Coleções no JDK 1.2

- Classe Hashtable

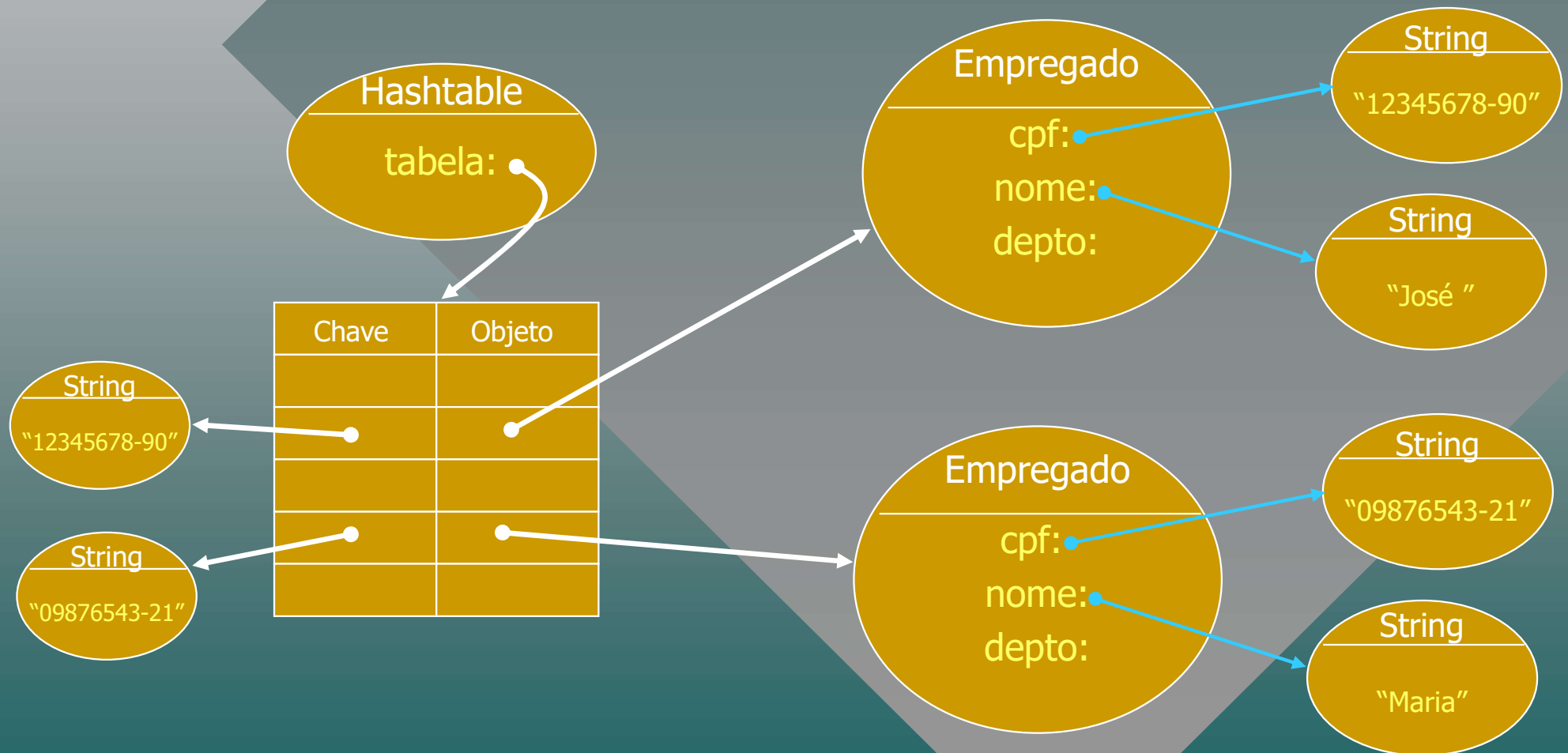
- Não está vinculada a interface *Collection* (implementa na realidade a interface *Map*)
- Funciona como uma tabela Hash. Para adicionarmos um elemento, temos que vincular a ele uma chave de indexação que é utilizada para a recuperação.
 - `public void put(Object chave, Object elemento)`
 - Insere um elemento no Hashtable indexando-o pela chave passada.
 - `public Object get(Object chave)`
 - Recupera o elemento indexado no Hashtable pela chave fornecida.
- A posição de um elemento é determinada por sua função hash. Em Java, todo objeto possui o método `hashCode()` e este é utilizado como função hash.
- Como Hashtable é *synchronized*, é melhor utilizar a classe *HashMap*.



Hashtable

Exemplo

- Se enviarmos a mensagem `get("12345678-90")` para o Hashtable, ele nos dará uma referência para o objeto Empregado "José".



Coleções no JDK 1.2

– Outros Métodos Importantes de Hashtable:

- `Hashtable()`
- `Hashtable(int capacidadeInicial)`
- `Hashtable(int capacidadeInicial, float fatorDeCarga)`
- `boolean containsValue(Object o)`
- `boolean containsKey(Object o)`
- `Set keySet()`
- `Collection values()`
- `Object remove(Object key)`

– **Hashtable** é uma classe synchronized. Prefira a classe **HashMap**.